BIO-INSPIRED ARTIFICIAL INTELLIGENCE

Evolutionary Algorithms I

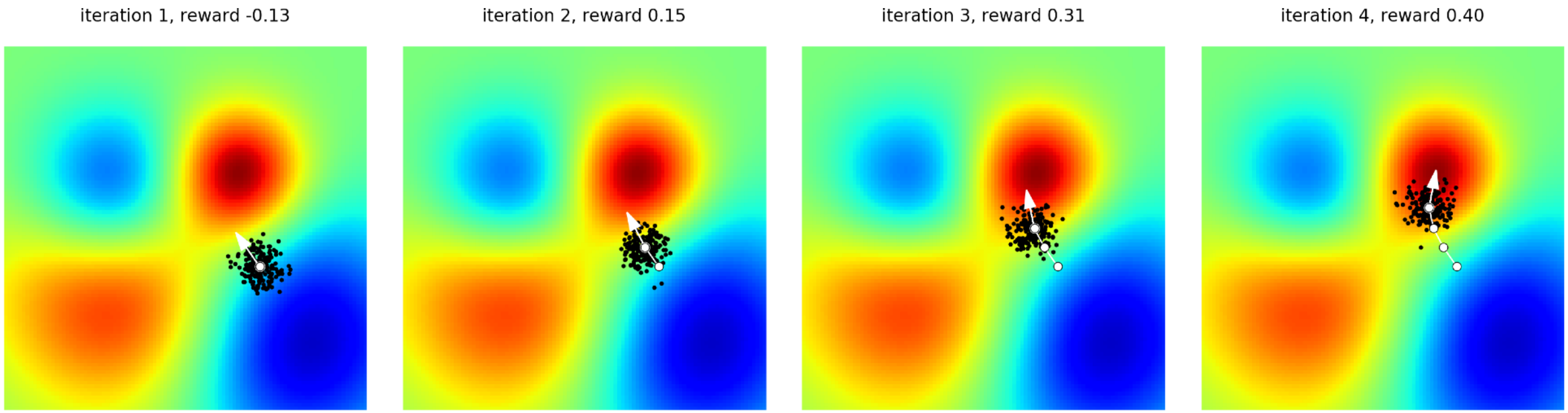
Prof. Giovanni Iacca

giovanni.iacca@unitn.it

UNIVERSITY OF TRENTO - Italy

Information Engineering
and Computer Science Department

Genetic Algorithms



GENETIC ALGORITHMS

EVOLUTIONARY ALGORITHMS: CHECK LIST

1. Devise genetic representation
2. Build a population
3. Design a fitness function
4. Choose selection method
5. Choose replacement method
6. Choose crossover & mutation
7. Choose data analysis method
8. Repeat generation cycle until
   - maximum fitness value is found
   - solution found is "good enough"
   - maximum wall-clock time is reached
   - a certain convergence condition is met

Evolutionary algorithms are applicable to any problem domain as long as suitable genetic representation, fitness, and genetic operators are chosen.

GENETIC ALGORITHMS

GENETIC REPRESENTATION

- Describes elements of genotype and mapping to phenotype
- Must match genetic operators of recombination and mutation
- Set of possible genotypes should include optimal solution to the problem

- Choice of representation benefits from domain knowledge:
  - Encoding of relevant parameters
  - Appropriate accuracy
  - Completeness: in order to find the global optimum, every feasible solution must be represented in genotype space

- Great simplification of genetics:
  - Single-stranded sequence of symbols (e.g., binary)
  - Often fixed-length, only genic "DNA"
  - Often one chromosome, with haploid structure (one copy only of the chromosome)
  - Often one-to-one direct correspondence between gene and parameter
  - Gene expression and genetic regulation used only in specific situations

GENETIC ALGORITHMS

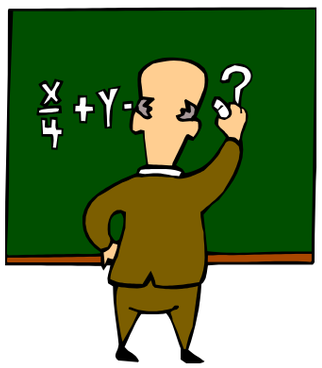
DISCRETE REPRESENTATIONS

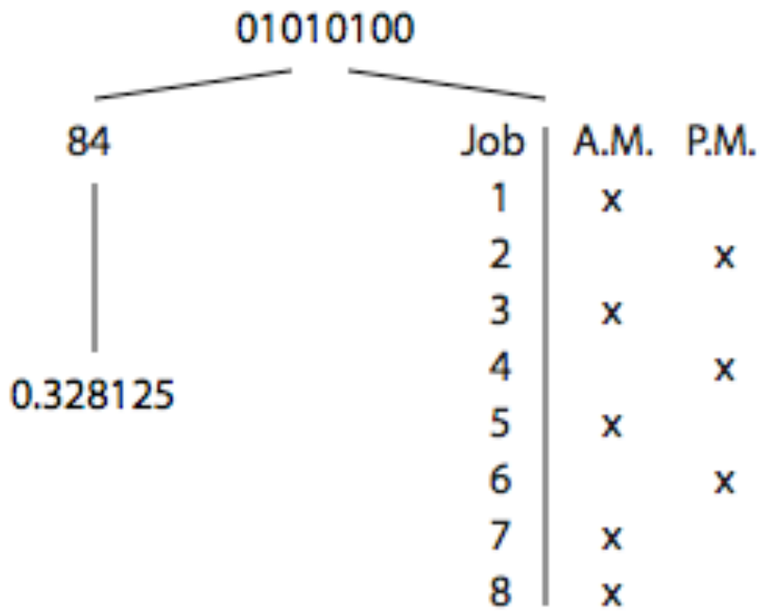
The genotype is a sequence (or an array of sequences) of *n* discrete symbols drawn from alphabet with cardinality *k*.

E.g., a binary string of 8 digits (*n*=8, *k*=2), such as 01010100, can be mapped into different phenotypes, depending on the optimization problem (just to give some examples):

1. To integer *i* using binary code

2. To real value *r* in range [*min*, *max*]:
r = *min* + (*i*/255)(*max-min*)





3. A binary assignment, such as a job schedule problem
- job=gene position
- time=gene value

GENETIC ALGORITHMS

DISCRETE REPRESENTATIONS

Problems with using a binary representation:

- Conversion form a real value to a bitstring of *n* bits has a maximum accuracy (*max-min*)/(2^n-1)
  E.g., if *max*=1.0, *min*=0.0, n=8 bits → accuracy = 1.0/255~0.0039
- Using more bits → better accuracy but longer chromosomes (slow evolution)
- Binary coding introduces "Hamming cliffs": when two numerically adjacent values have bit representations that are far apart.
  E.g. $7_{10} = 0111_2$ vs $8_{10} = 1000_2$ → Large change in solution is needed for small change in fitness!

Gray coding

- Minimum Hamming distance between representations of two successive numerical values (distance=1).
- Given a bitstring: $B=[b_1 b_2 \ldots b_n]$, the corresponding Gray bitstring $G=[g_1 g_2 \ldots g_n]$ can be obtained as follows:
  - $g_1 = b_1$
  - $g_k = (b_{k-1}$ AND (NOT b_1)) OR ((NOT b_{k-1}) AND b_1)
  where b_k is the k-th bit of the bitstring.

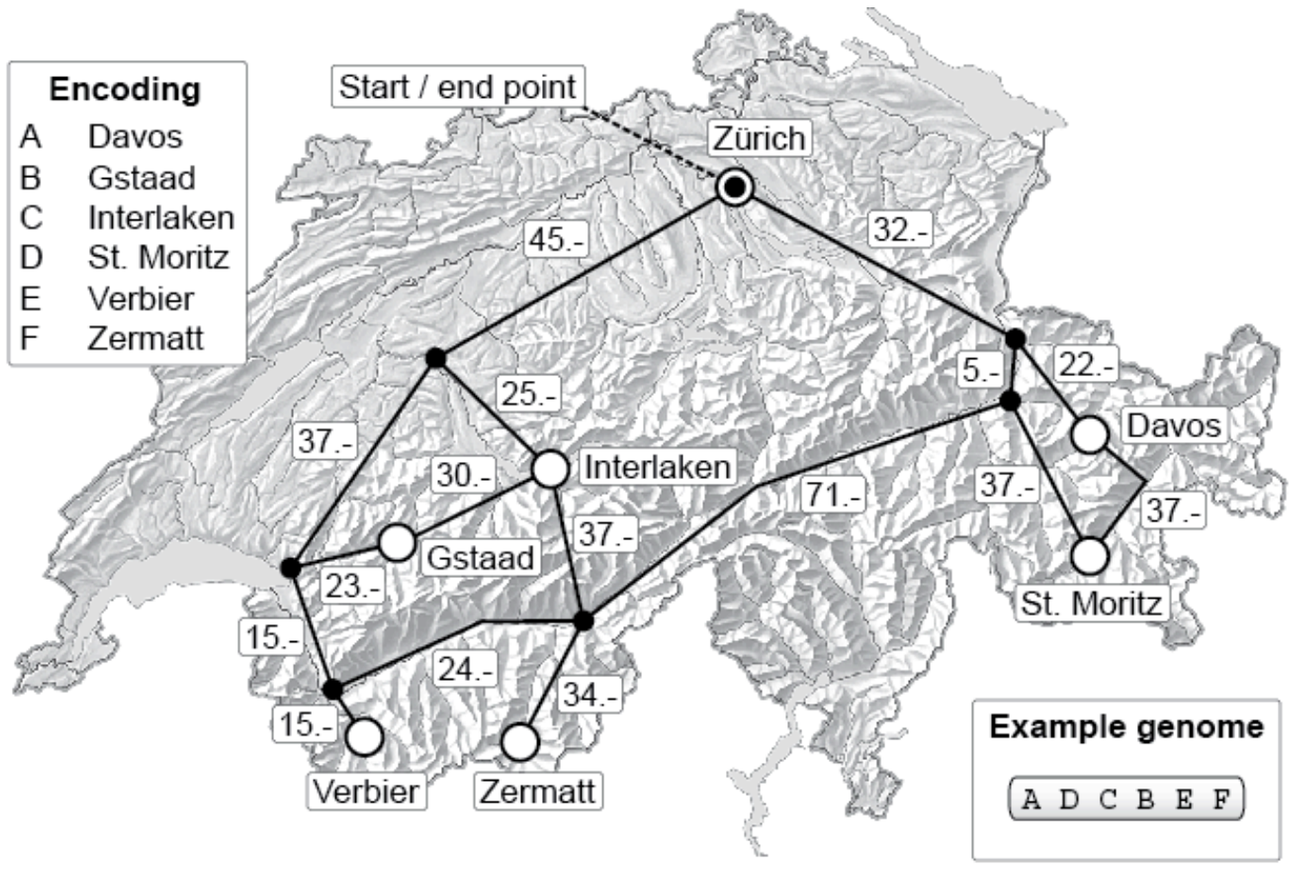
	Binary	*Gray*
0	000	000
1	001	001
2	010	011
3	011	010
4	100	110
5	101	111
6	110	101
7	111	100

GENETIC ALGORITHMS

SEQUENCE REPRESENTATIONS

It is a particular case of discrete representation used e.g. for Traveling Salesman Problems (TSP), i.e., plan a path to visit *n* cities under some constraints, and problems alike. In this case the individual is a permutation of *n* different symbols (e.g. integers, letters, or labels), each of which occurs exactly once.

E.g., planning ski holidays with lowest transportation costs:

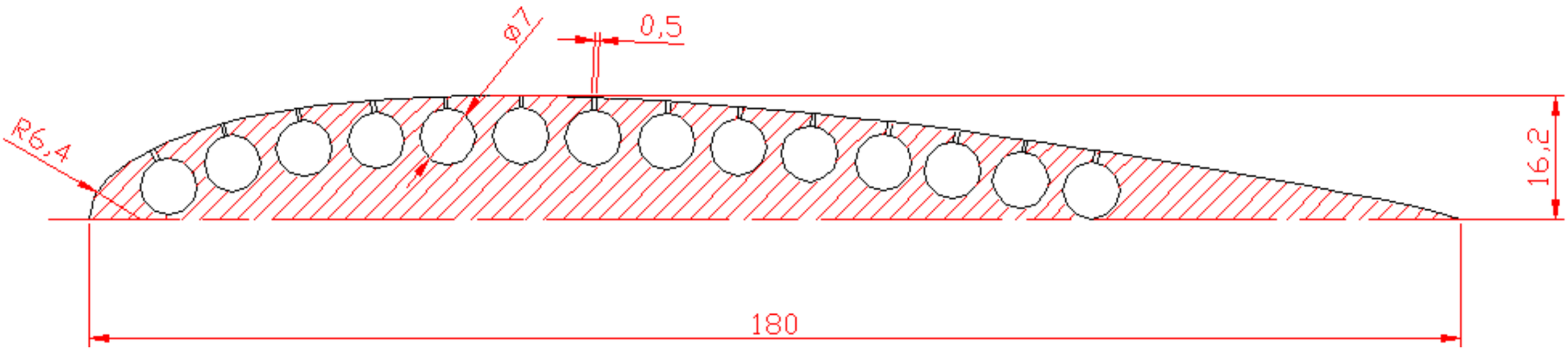


GENETIC ALGORITHMS

REAL-VALUED REPRESENTATIONS

The genotype is a sequence of real values that represent the problem parameters.

- Used when high-precision parameter optimization is required
- For example, genetic encoding of wing profile for shape optimization:



Evolvable wing made of deformable material with pressure tubes. In this case the genotype is a vector representing the pressure values of 14 tubes.

GENETIC ALGORITHMS

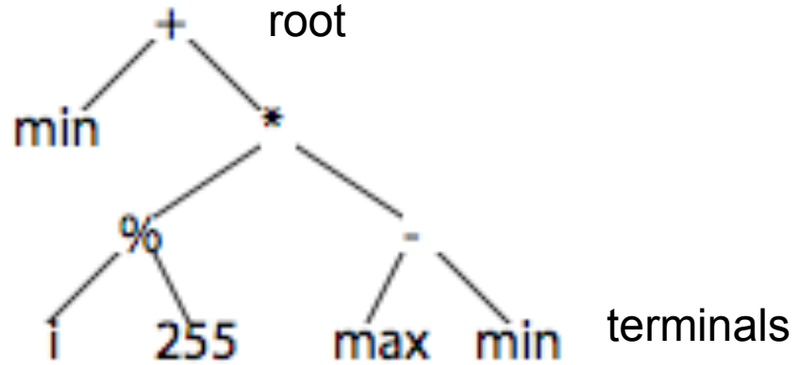
TREE REPRESENTATIONS

The genotype describes a tree with branching points and terminals.

- Suitable for encoding hierarchical structures
- E.g., used to encode computer programs (Genetic Programming, GP) made of:
  - Operators ("Function set": multiplication, If-Then, Log, etc.)
  - Operands ("Terminal set": constants, variables, sensor readings, etc.)

Expression r=min+(i/255)(max-min)

Nested list (+, min, (*, (/, i, 255), (-, max, min)))



REQUIREMENTS FOR GP

- Closure: all the "encodable" functions must accept as arguments all terminals in Terminal set, and all possible outputs of all functions in Function set (e.g., protected division %)
- Sufficiency: elements in Function and Terminal sets must be sufficient to generate the program that solves the given problem

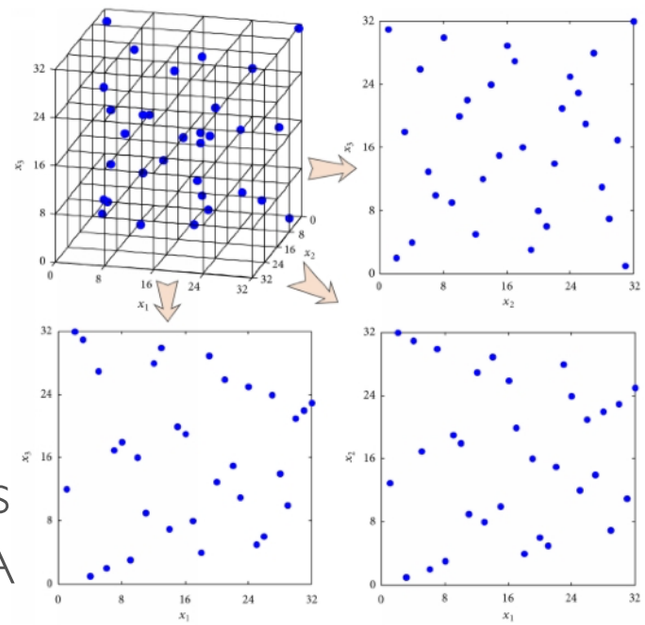
GENETIC ALGORITHMS

INITIAL POPULATION

Should be sufficiently large to cover the search space (!), but sufficiently small in terms of evaluations (typical size: between 10s and 1000s individuals).

Uniform sample of search space:

- Binary strings: 0 or 1 with probability 0.5 (similarly for other discrete representations)
- Real-valued representations: uniform on a given interval if bounded phenotype (e.g., +2.0, -2.0), otherwise "best guess" (based on domain knowledge). Alternatively, use a DoE (Design of Experiments) algorithm.
- Trees are built recursively starting from root: root is randomly chosen from Function set; for every branch, randomly choose among all elements of Function set and of Terminal set; if terminal is chosen, it becomes leaf. A maximum tree depth must be chosen.



IMPORTANT: hand-designed genotypes may cause a loss of genetic diversity (see later) and/or an unrecoverable bias in the evolutionary process!

GENETIC ALGORITHMS

FITNESS FUNCTION = OBJECTIVE FUNCTION

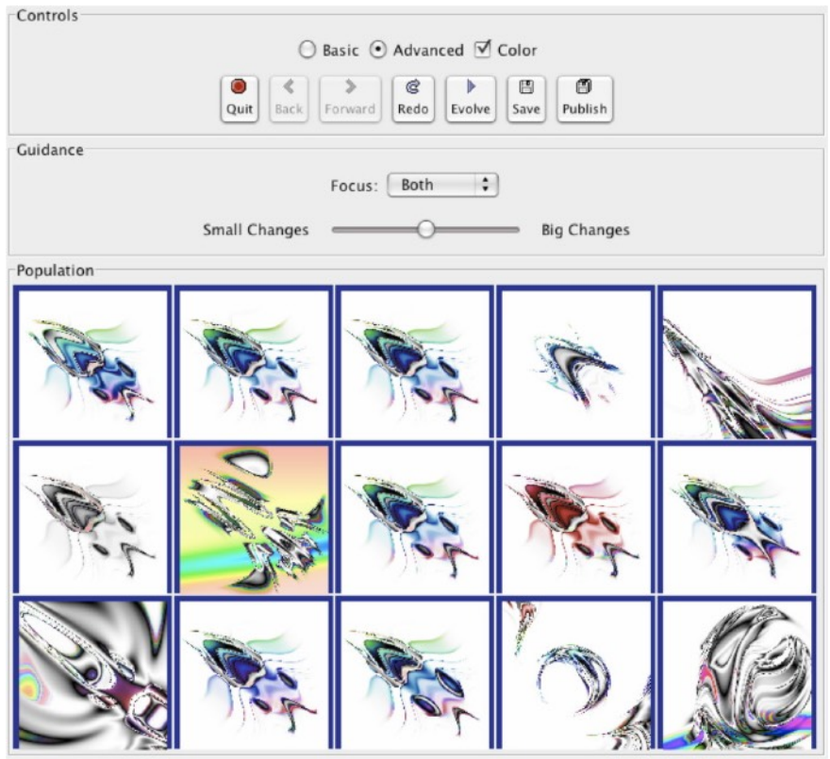
Evaluates the performance of a phenotype with (one or more) numerical scores.

- Choice of components; e.g., lift and drag of a wing
- Combination of components; e.g. (lift + 1.0/drag) or (lift - drag)
- Extensive test of each phenotype (with noise, repeated evaluations are needed)
- The more the fitness function can discriminate (return different values), the better
- Warning! "You Get What You Evaluate"

AN INTERESTING CONCEPT

Subjective fitness: select phenotype by human inspection (basis of IEC, Interactive Evolution Computation)

- Used when aesthetic properties cannot be quantified objectively (e.g. visual art, music, design, etc.)
- Can be combined with an objective fitness function
- E.g., Picbreeder http://picbreeder.org/



GENETIC ALGORITHMS

(PARENT) SELECTION

A method to make sure that better individuals make comparatively more offspring.

- Used in breeding (pedigreed dogs, horses, etc.)
- **Selection pressure** is inversely proportional to no. of selected individuals
- High selection pressure = rapid loss of diversity and premature convergence
- Important that also less performing individuals can reproduce to some extent

GENETIC ALGORITHMS

RANDOM SELECTION

- Each individual has a probability of 1/*N* to be selected, where *N* is the population size
- No fitness information is used → actually, never used in EA!
- Lowest selection pressure among all possible selection schemes

GENETIC ALGORITHMS

FITNESS-PROPORTIONATE SELECTION (ROULETTE-WHEEL SELECTION)

The probability that an individual makes an offspring is proportional to how good its fitness is with respect to the population fitness: $p(i) = f(i)/\Sigma f(i)$, i is the individual index
→ **Biases selection towards the most-fit individuals!**

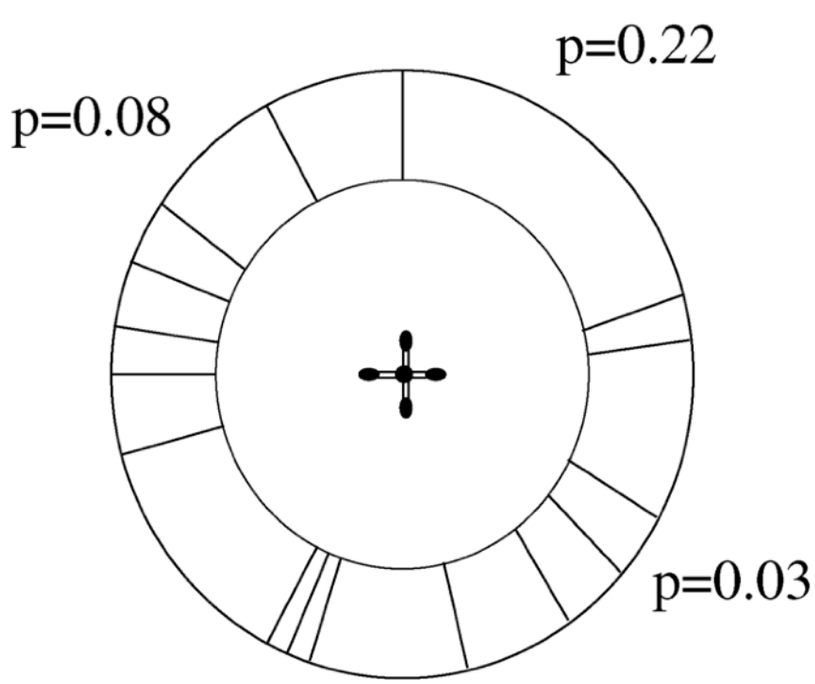
Problems:
- Fitness must be non-negative (otherwise must be shifted to positive values)
- Uniform fitness values (values are too close) = random selection
- Few high-fitness individuals = high selection pressure (low-fitness individuals have close-to-zero chance of reproduction)

GENETIC ALGORITHMS

RANK-BASED SELECTION

- Individuals are sorted on their fitness value from best to worst. The place in this sorted list is called the **rank** *r* (rank=1 for best, rank=N-1 for worst, being N the population size).
- Instead of using the fitness value of an individual, the rank is used to select individuals:
  $p(i) = 1\text{-}r(i)/\Sigma r(i)$ ⟶ lower selection pressure w.r.t. fitness-proportionate selection
- Roulette wheel on rank-based $p(i)$

individual	fitness	rank
A	5	5
B	7	3
C	8	2
D	2	8
E	3	7
F	9	1
G	7	4
H	4	6



GENETIC ALGORITHMS

TRUNCATED RANK-BASED SELECTION

- Only the best *x* individuals are allowed to make offspring and each of them makes the same number of offspring: *N*/*x*, where *N* is the population size.
- E.g., in a population of 100 individuals, make 5 copies of 20 best individuals

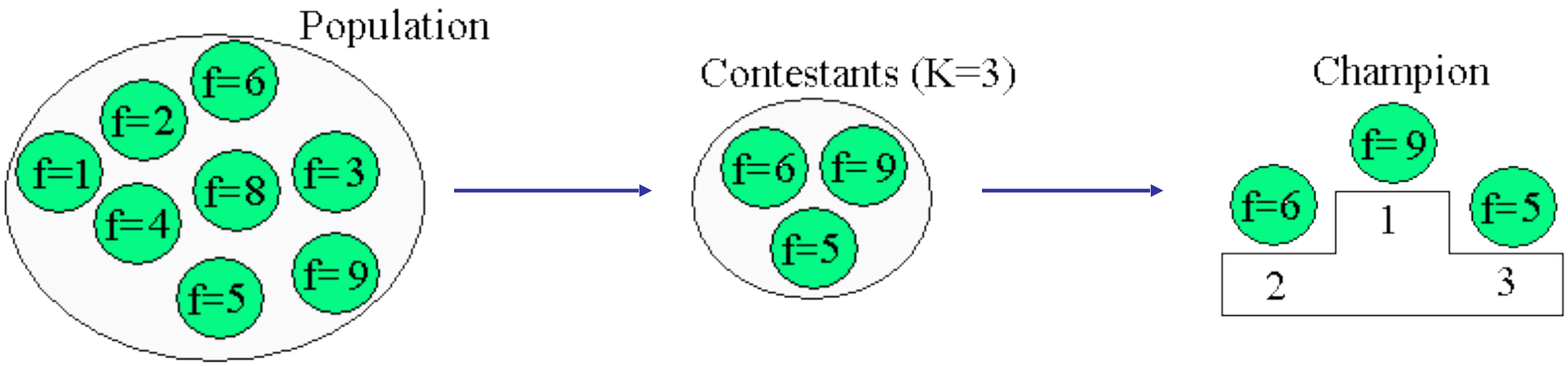
individual	fitness	rank	list
A	5	5	F
B	7	3	C
C	8	2	B
D	2	8	G
E	3	7	A
F	9	1	H
G	7	4	E
H	4	6	D

GENETIC ALGORITHMS

TOURNAMENT SELECTION

For every offspring to be generated:

1. Pick randomly *k* individuals from the population, where *k* is the tournament size < *N*, *N* being the population size (larger k = larger selection pressure, i.e., individual have a higher chance to compete against individuals with higher fitness)
2. Choose the individual with the highest fitness and make a copy
3. Put all individuals back in the population



GENETIC ALGORITHMS

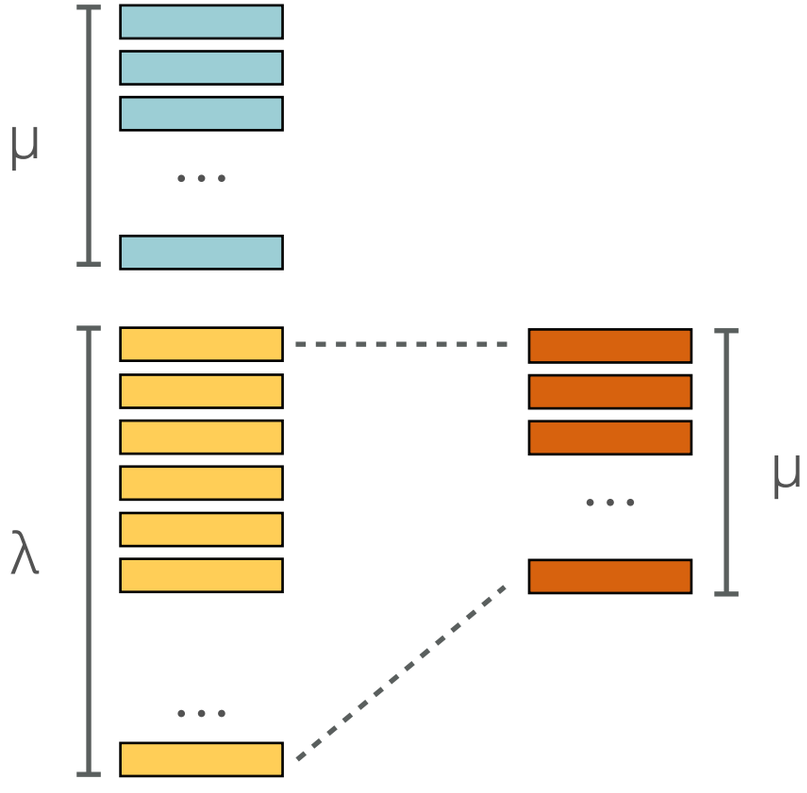
"HALL OF FAME" SELECTION

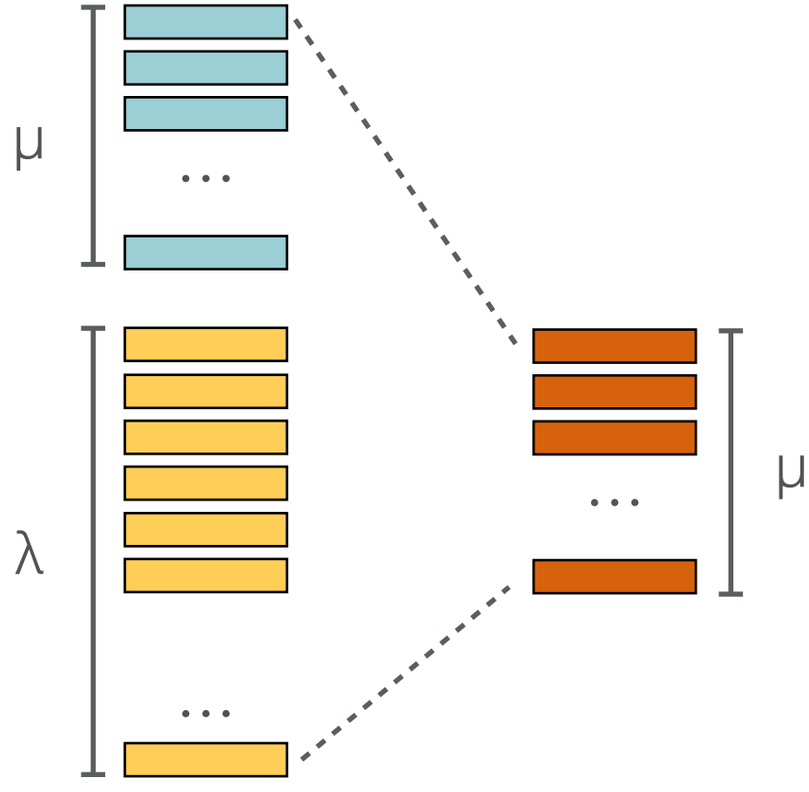
- Similar to the list of best players of an arcade game
- For each generation, the best individual is selected to be inserted into the hall of fame
- Contain an archive of the best individuals found from the first generation
- The hall of fame can be used as a parent pool for the crossover operator (see later)
- Especially used in co-evolutionary algorithms

GENETIC ALGORITHMS

(μ,λ) or (μ+λ) SELECTION (A.K.A. "COMMA" or "PLUS" STRATEGIES)

- Deterministic rank-based selection methods (mostly used in Evolution Strategies)
- μ indicates the number of parents
- λ indicates the number of offspring produced (λ≥μ), i.e., λ/μ offspring per parent
- (μ, λ)-selection selects the best μ individuals from λ offspring
- (μ+λ)-selection selects the best μ individuals from both μ parents λ offspring





GENETIC ALGORITHMS

REPLACEMENT (OR "SURVIVOR SELECTION")

- **Generational replacement**: old population is entirely replaced by offspring (also called "aged-based" replacement, or "non-overlapping" replacement, most frequent method)

- **Generational rollover**: insert offspring "in place", i.e., replace worse individuals in the current generation. A special case is the **Steady-State scheme** (Whitley et al., 1988): one offspring is generated per generation, one member of the population (the worst one) is replaced.

- **Elitism**: maintain *n* best individuals from previous generation to prevent loss of best individuals by effects of mutations or sub-optimal fitness evaluation (the higher *n*, the less diversity)

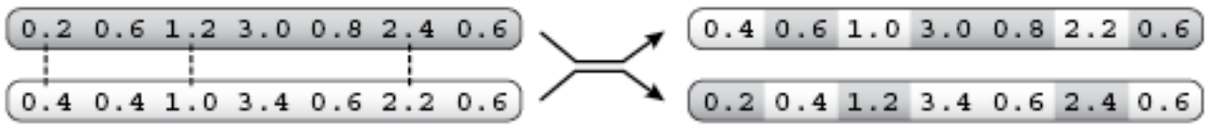
GENETIC ALGORITHMS

RECOMBINATION (CROSSOVER)

- Emulates recombination of genetic material from two parents during meiosis
- **Exploitation** of synergy of sub-solutions (building blocks) from parents
- Applied to randomly paired offspring with a given probability *Pc*

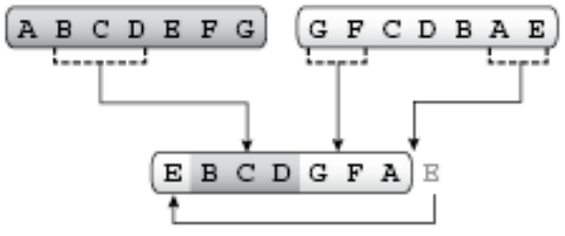
One point

Uniform

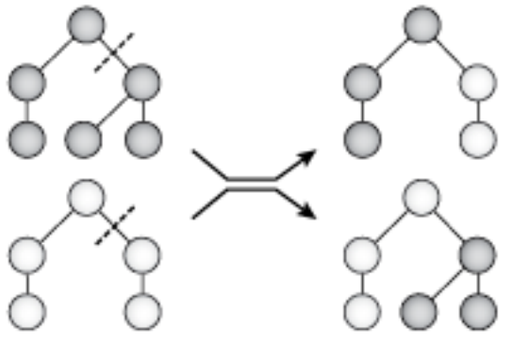


Arithmetic

For sequences



For trees

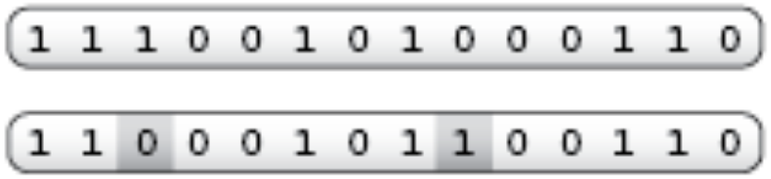


GENETIC ALGORITHMS

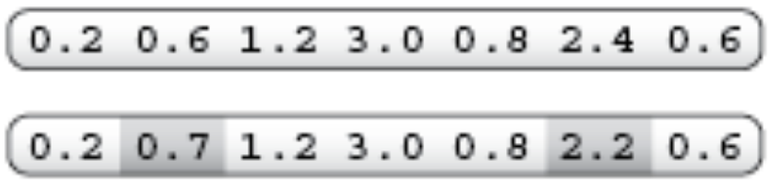
MUTATION

- Emulates genetic mutations
- **Exploration** of variation of existing solutions
- Applied to each gene in the genotype with a given probability *Pm*

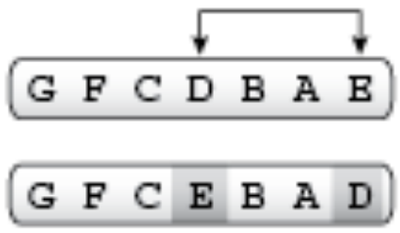
Binary genotypes



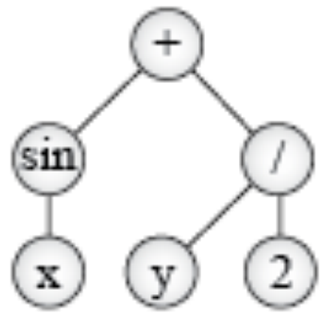
Real-valued genotypes

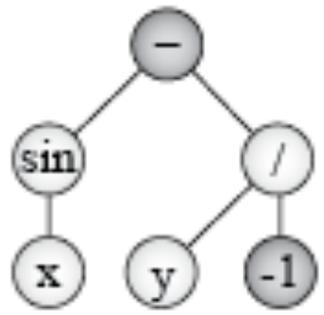


Sequence genotypes



For trees

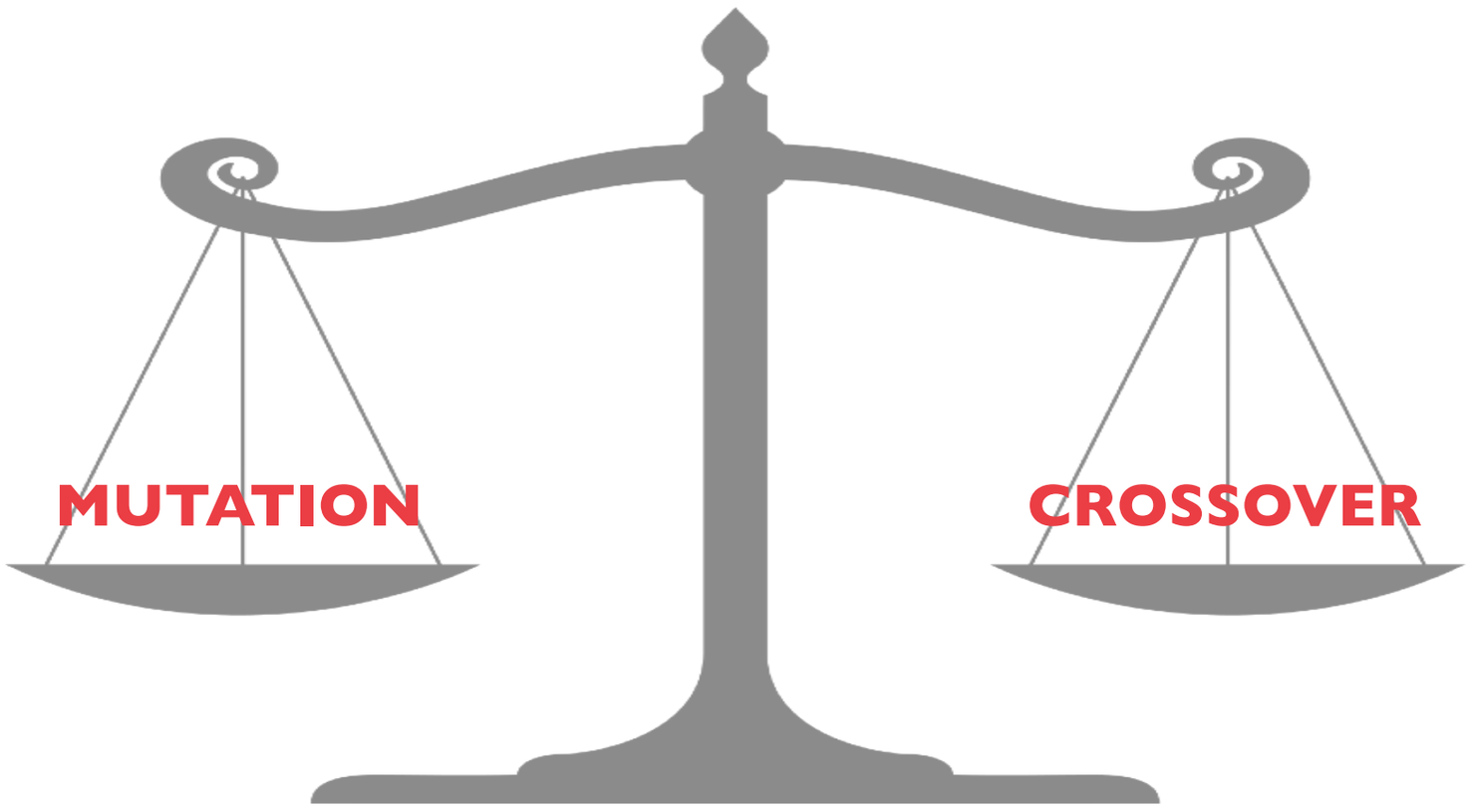




GENETIC ALGORITHMS

CROSSOVER OR MUTATION?

- A long debate: which one is better/necessary?
- Answer (at least, rather wide agreement): it depends on the problem, but, in general, it is good to have both! Mutation provides exploration, while crossover provides exploitation capabilities.
- While mutation-only-EA is possible, crossover-only- EA would not work (no introduction of new genetic material in the population).



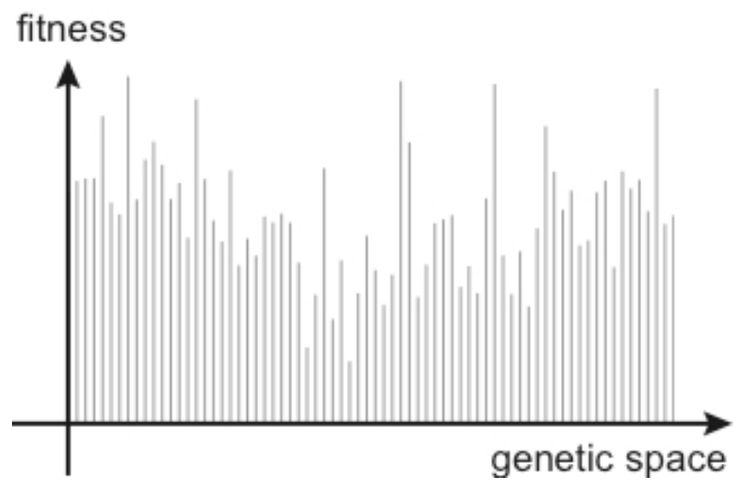
GENETIC ALGORITHMS

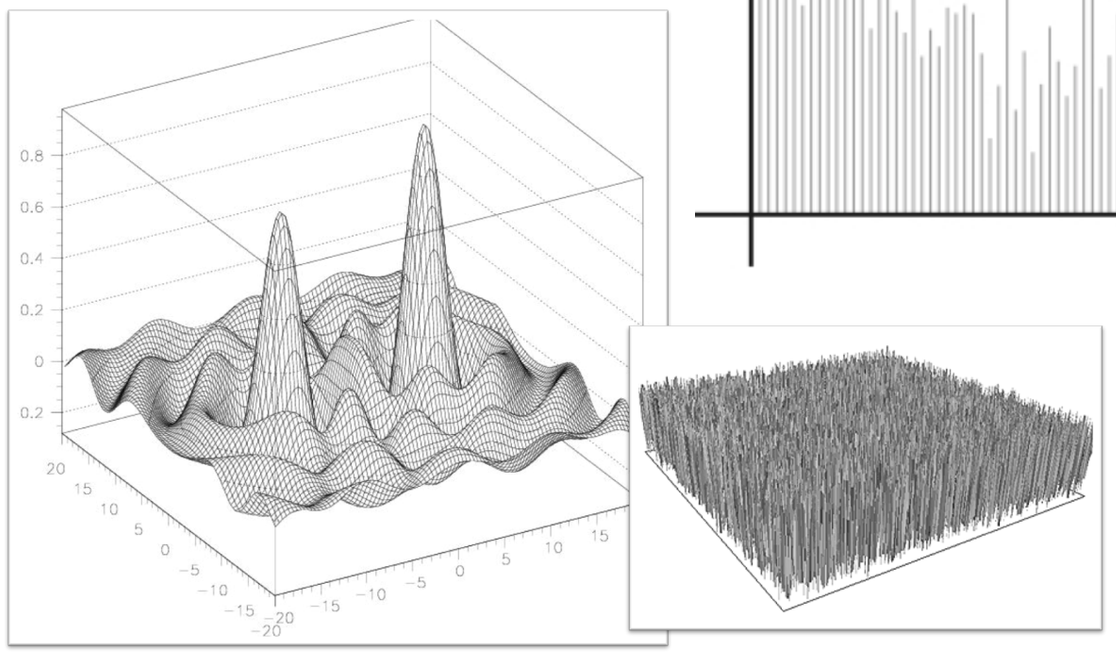
DATA ANALYSIS: ASSESSING FITNESS LANDSCAPE

- Fitness landscape is a plot of fitness values associated to all genotypes
- Real landscape is unknown; estimation helps to assess evolvability
- Goal of evolution is to find genotype with best fitness
- "Navigation" based on genetic operators → landscape metaphor somehow misleading

Estimating "ruggedness" of a fitness landscape:

1. Sample random genotypes: if flat, use large populations
2. Explore surroundings of individual by applying genetic operators in sequence for a fixed number of times: the larger the fitness improvement, the "easier" is to evolve





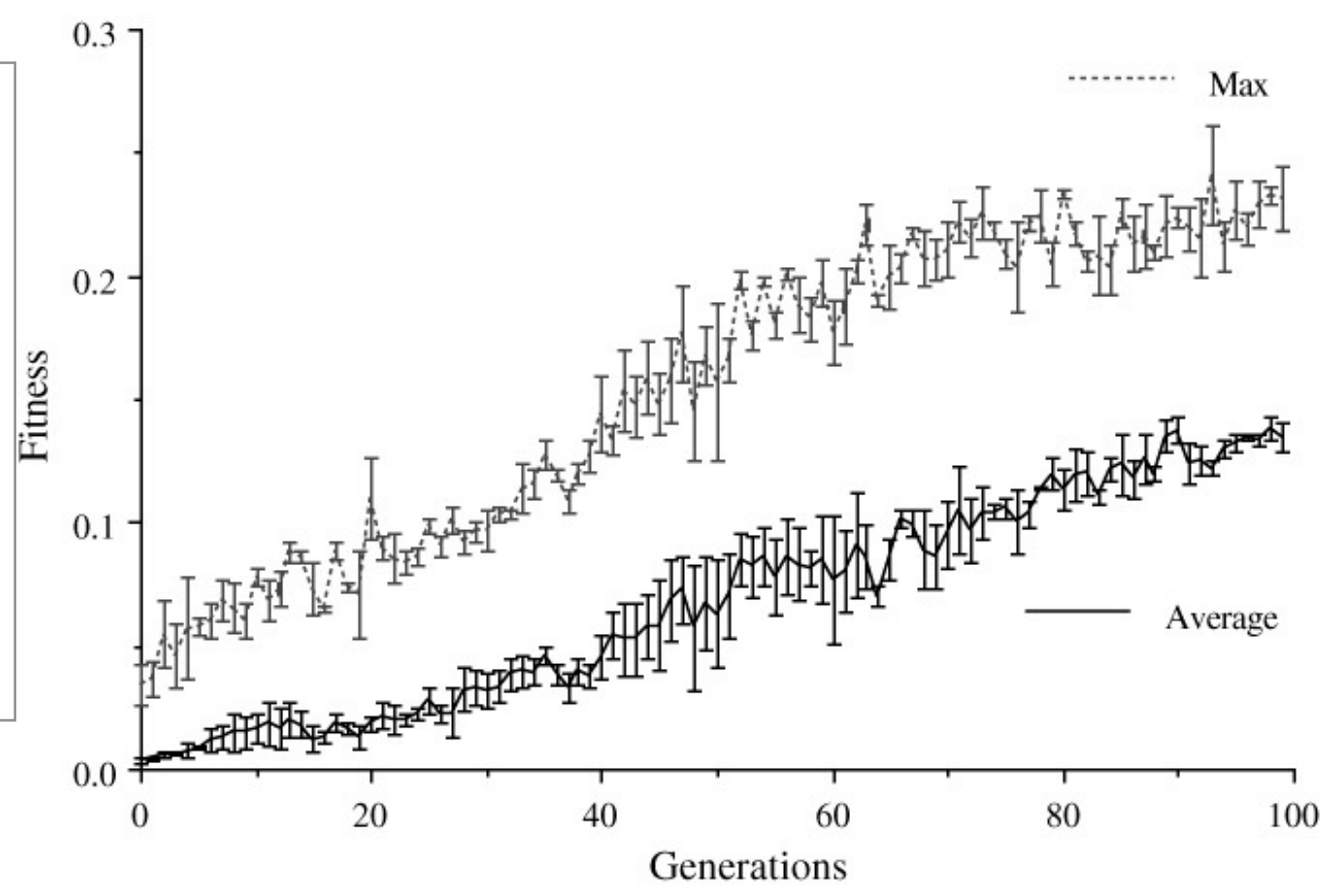
GENETIC ALGORITHMS

DATA ANALYSIS: MONITORING PERFORMANCE

- Track best/worst and/or population average fitness (+/- std. dev.) of each generation
- Multiple runs are necessary ⟶ plot average data and standard error ("anytime behavior")
- Fitness graphs (also called "fitness trends") are meaningful only if the problem is stationary, i.e., the fitness function does not change over time
- These plots can be used to detect if the algorithm stagnated or (prematurely) converged

Stagnation: no further evolution possible even if the population remains diverse (individuals are far from each other, but new individuals are worse).

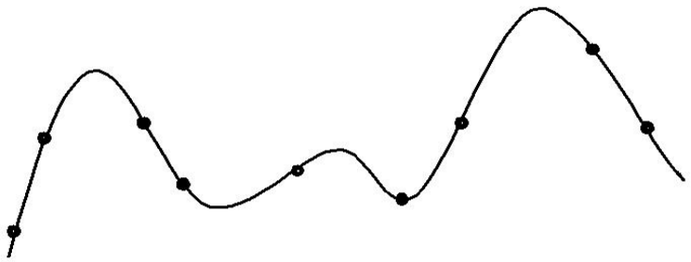
Premature convergence: no further evolution possible since the population lost diversity (individuals are too close to each other.)



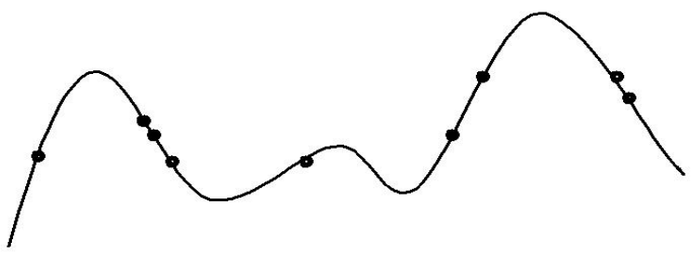
GENETIC ALGORITHMS

DATA ANALYSIS: MONITORING PERFORMANCE

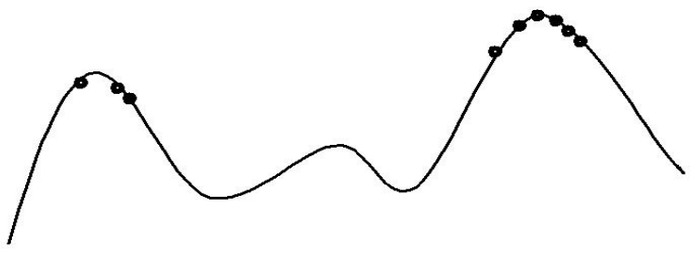
- What is the typical behavior of an EA?
- Can we identify some phases in an evolutionary run?



Early phase:
quasi-random population distribution



Mid-phase:
population arranged around/on hills



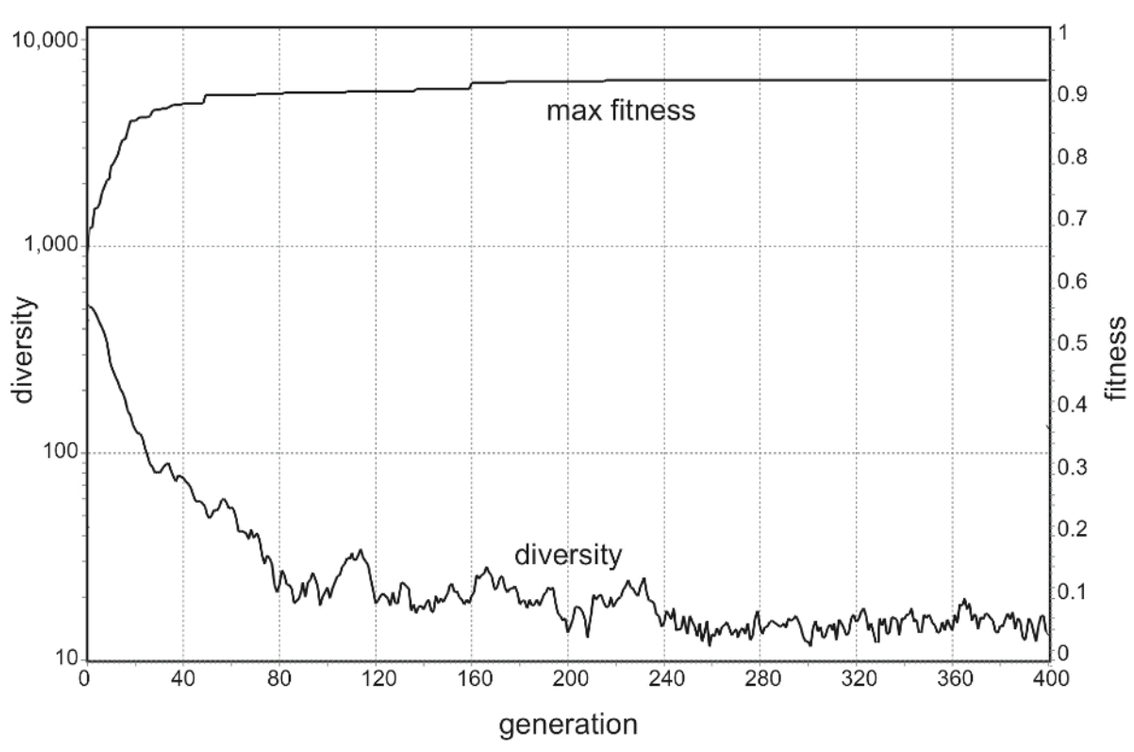
Late phase:
population concentrated on high hills

GENETIC ALGORITHMS

DATA ANALYSIS: MEASURING DIVERSITY

- Diversity tells whether the population has potential for further evolution
- Measures of diversity depend on genetic representation
- E.g., for real and discrete values, sum of Hamming and Euclidean distances, respectively:

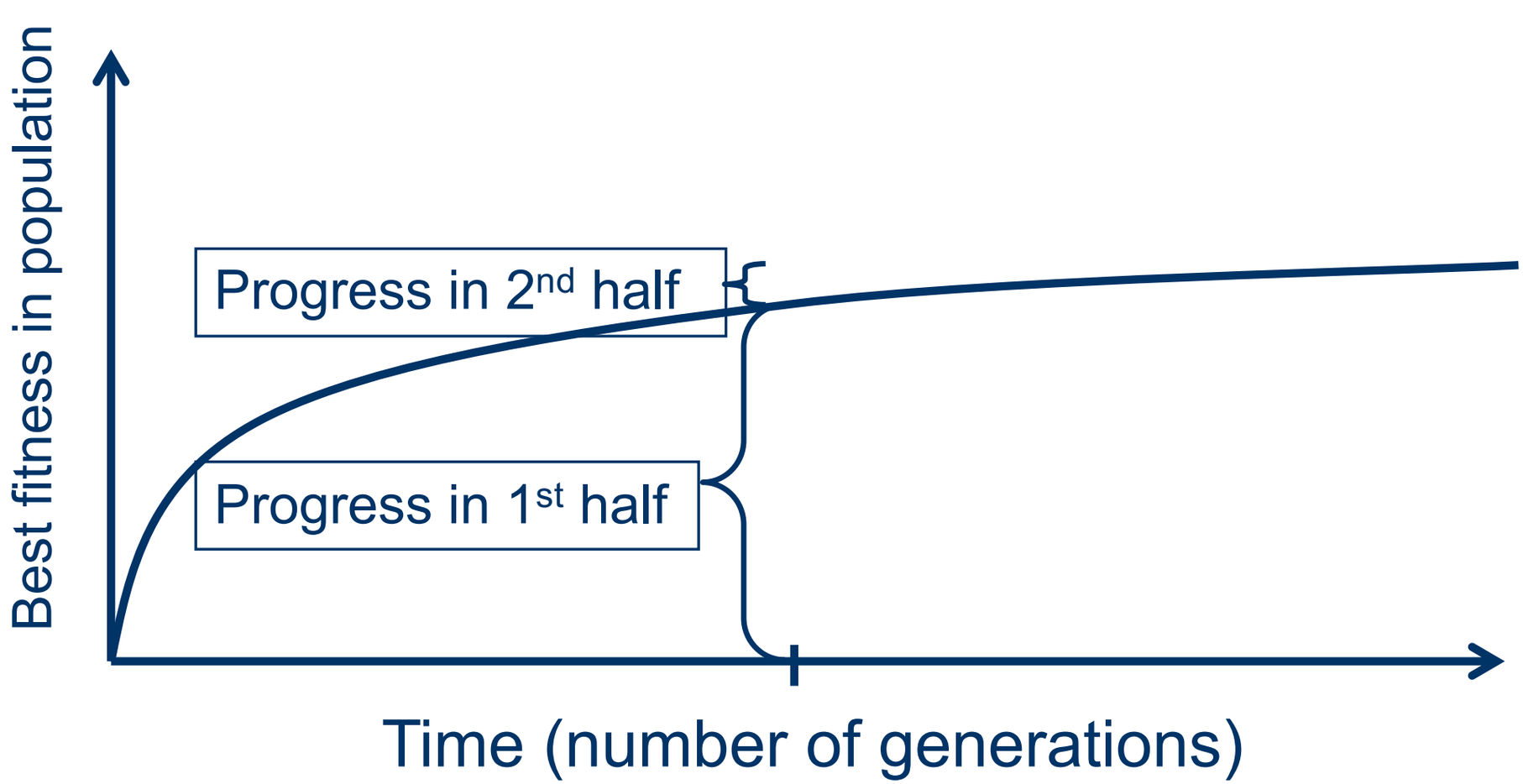
$$D_a(P) = \sum_{i,j \in P} d(g_i, g_j)$$



GENETIC ALGORITHMS

SOME EXTRA CONSIDERATIONS

- Is it worth putting effort on smart initialization?
  - It depends. Yes, if good solutions/methods exist, can be used to seed the initial generation with "super-fit" individuals and give the algorithm a sort of "head start"
- Are "long" runs beneficial?
  - It depends on how much one wants the last bit of progress
  - It may be better to do multiple shorter runs



GENETIC ALGORITHMS

GENERALITIES

- The original GA is now known as the **Simple Genetic Algorithm** (Holland, 1975)

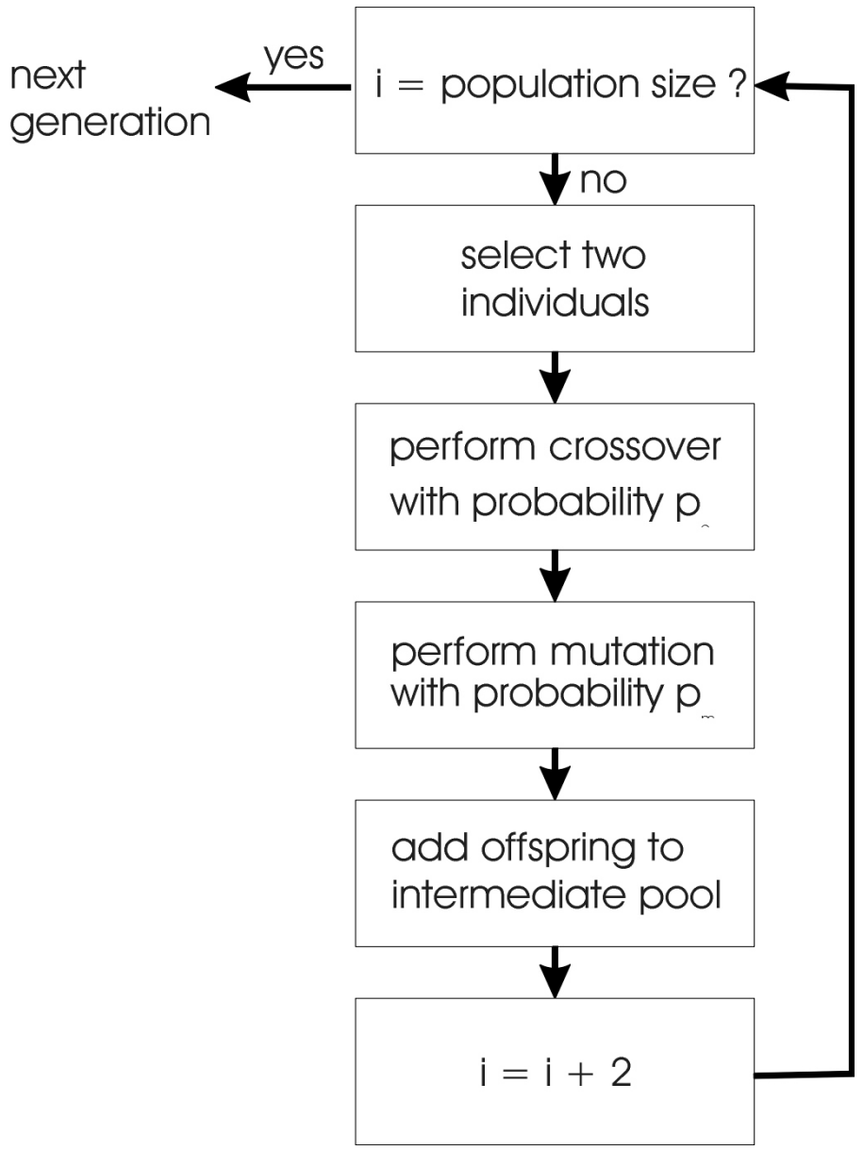
- Main features
  - Binary encoding, typically used for discrete optimization
  - However, binary encoding also used for real-valued problems, originally chosen to remark the analogy with biology (DNA: nucleotides→genes = bits → real values)
  - SGA: fitness-proportionate selection, uniform mutation, one-point cross-over
  - Now many variants of GAs: different mutation/crossover operators, selection models, etc.

NOTE

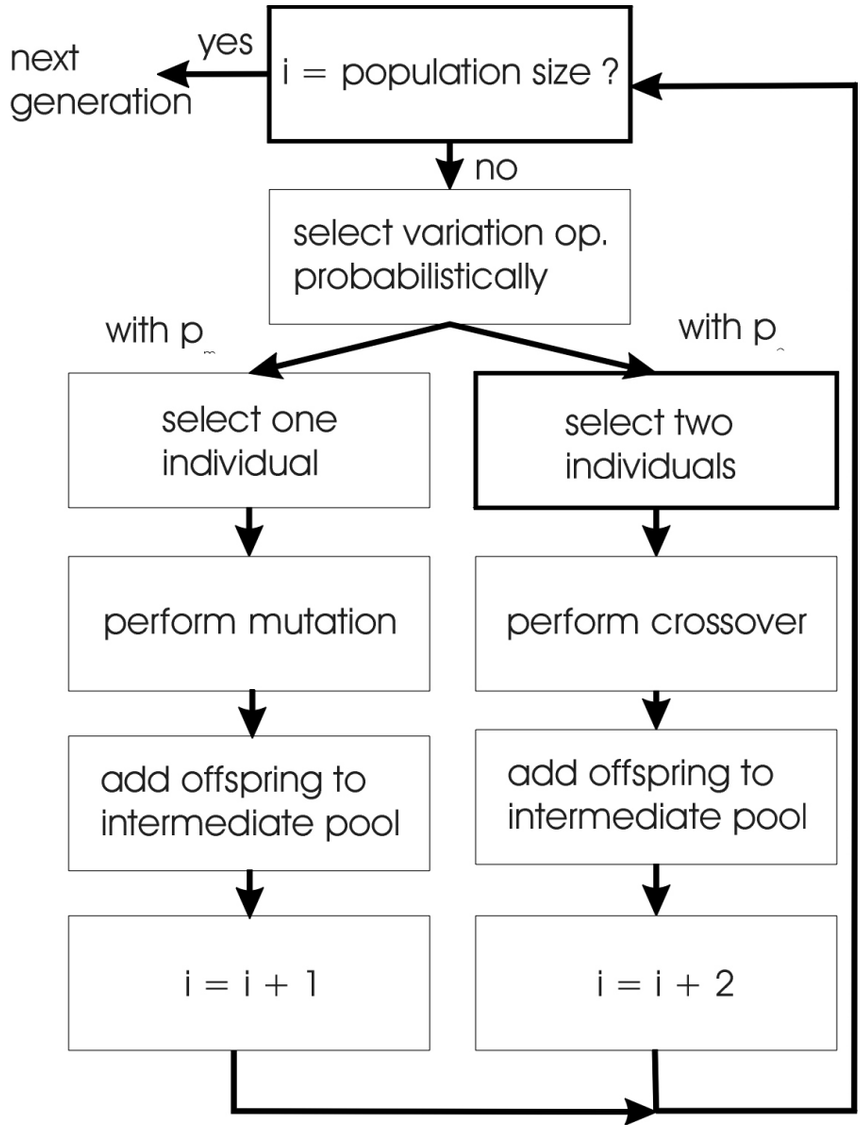
The SGA is nowadays generally considered rather inefficient, especially for continuous optimization (in that case there is no real need to discretize the problem), and should never be applied unless the problem is naturally binary.

GENETIC ALGORITHMS

ALTERNATIVE TYPES OF GENETIC ALGORITHMS



Recombination AND mutation



Recombination OR mutation

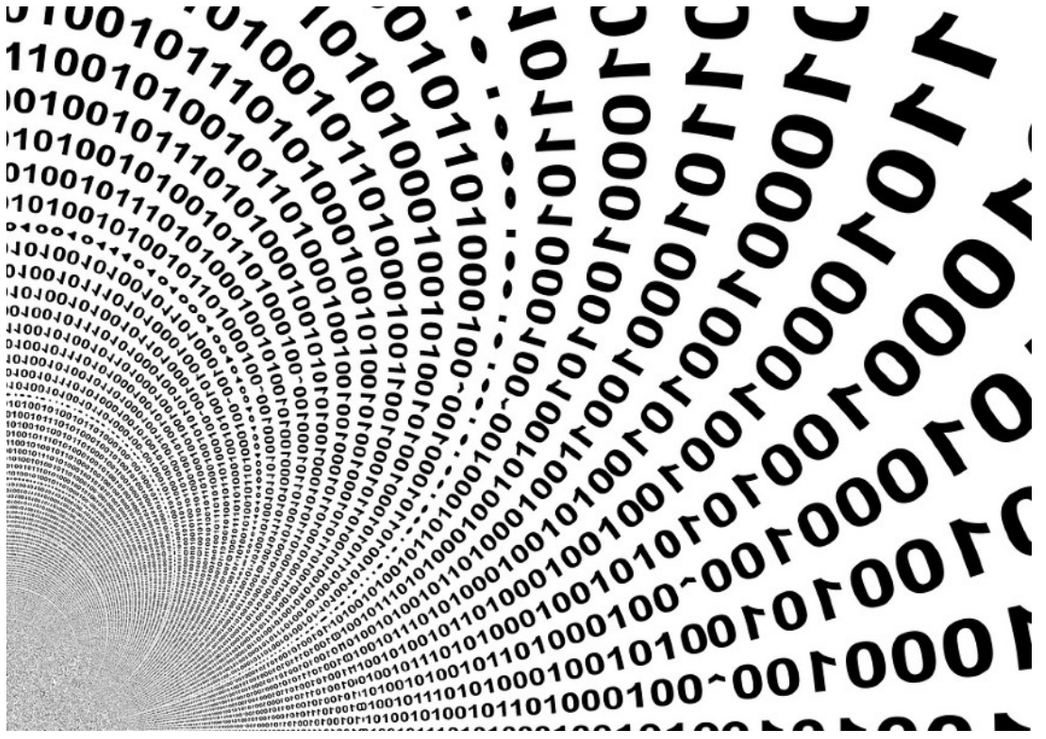
GENETIC ALGORITHMS

EXAMPLE: ONE-MAX

- Simple benchmark binary problem:
  maximize $f(x)=x_1+x_2+\ldots+x_n$
  where x_i in {0,1} for each i in [1,n]

  Optimum $x^* = [1\ 1\ 1\ \ldots\ 1] \rightarrow f(x^*) = n$

- Individual representation = n-dimensional binary string
- Fitness = sum of 1's in the candidate solution
- GA configuration: generational, random initialization
- Selection: roulette wheel fitness (roulette wheel)
- Genetic operators:
  - crossover: 1-point
  - mutation: single bit flip, $Pm = 1/n$ ("golden rule")
- Stopping condition: reach the optimum

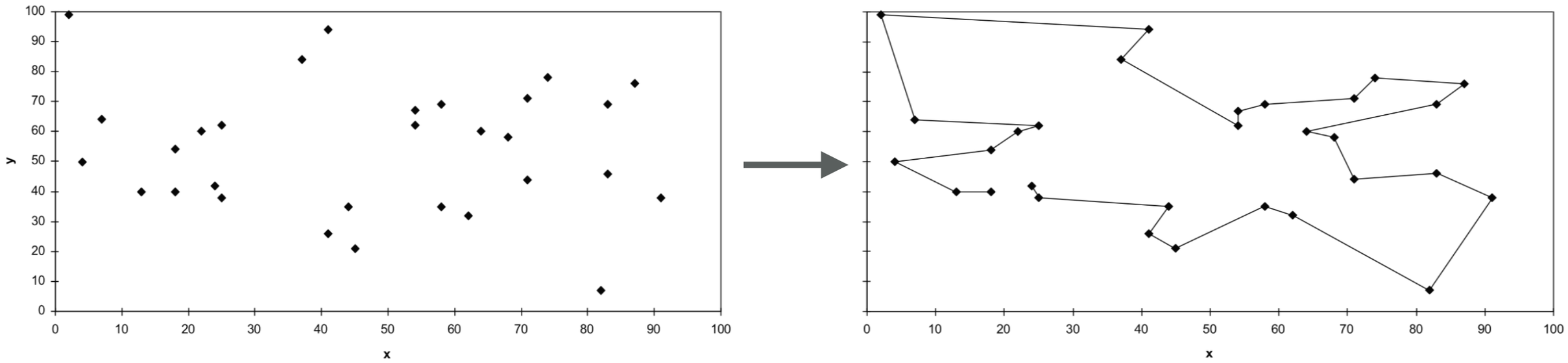


GENETIC ALGORITHMS

EXAMPLE: TRAVELING SALESMAN PROBLEM (TSP)

- Given a set of cities with relative distances, find the shortest possible route that visits each city exactly once and returns to the origin city (shortest Hamiltonian cycle)

- Individual representation = permutation of *n* symbols {A, B, C…} (each symbol=a city)
- Fitness = length of the route (sum of the relative distances of the sequence)
- GA configuration: generational, random initialization
- Selection: roulette wheel fitness (roulette wheel)
- Genetic operators:
  - crossover: "Inver-Over" (custom operator for TSP, Tao & Michalewicz, 1998)
  - mutation: permutation, e.g. p=(C, G, H, B, F, I, A, D, E)→p' = (C, G, I, F, B, H, A, D, E)
- Stopping condition: reach the optimum



GENETIC ALGORITHMS

WHY/HOW GENETIC ALGORITHMS WORK: SCHEMATA THEORY

- **Def.**: A *schema* (pl. *schemata*) is a set of individuals with some common genes (Holland, 1988)
  Example (with binary representation): (* * * 1 * * 0 0 * * 1)
  where * indicates a "don't care" gene and the 1/0 indicate the common genes in the scheme (their positions are called *defined positions*).

- **Def.**: The *order* of a schema is the number of defined positions
  Example: (* * * 1 * * 0 0 * * 1) → order = 4

- **Def.** The *defining length* of a schema is the longest length between two defined positions, including the initial one.
  Example: (* * * 1 * * 0 0 * * 1) → defining length = 7

- **Building Block Hypothesis**: a GA seeks near optimal performance through the juxtaposition of short, low-order, high-performance schemata, called the "building blocks" (Goldberg, 1989)

- Short, low-order, schemata receive exponentially increasing trials in subsequent generations of a genetic algorithm (Michalewicz, 1992)

- Nowadays, this theory has less credit than some decades ago

Questions?